

Recursion in Java

A Comprehensive Guide

Concepts · Examples · Patterns · Best Practices

Java Programming Series

Topics Covered	8 Core Sections
Code Examples	15+ Java Programs
Difficulty	Beginner → Advanced
Language	Java 17+

Table of Contents

- 1. Introduction to Recursion
- 2. How Recursion Works – The Call Stack
- 3. Base Case and Recursive Case
- 4. Classic Recursion Examples
 - 4.1 Factorial
 - 4.2 Fibonacci Sequence
 - 4.3 Sum of Natural Numbers
 - 4.4 Power Function
- 5. Recursion with Arrays & Strings
 - 5.1 Reverse a String
 - 5.2 Binary Search
 - 5.3 Array Sum
- 6. Advanced Recursion Patterns
 - 6.1 Tower of Hanoi
 - 6.2 Merge Sort
 - 6.3 Tree Traversal
 - 6.4 Backtracking (N-Queens)
- 7. Recursion vs Iteration
- 8. Common Mistakes & Best Practices

1 Introduction to Recursion

Recursion is one of the most powerful and elegant concepts in computer science. In Java, a method is said to be **recursive** if it calls itself — either directly or indirectly — as part of its own execution. This self-referential technique allows complex problems to be broken down into smaller, simpler sub-problems of the same nature.

Recursion mirrors many real-world phenomena and mathematical definitions. It is widely used in algorithms involving trees, graphs, divide-and-conquer strategies, dynamic programming, and many other areas.

Definition: A function f is recursive if its definition references itself. In Java, a recursive method calls itself with a modified argument that moves closer to a terminal (base) condition.

Simple Recursive Method Skeleton

```
public class RecursionDemo {

    // A recursive method always has:
    // 1. Base case - stops recursion
    // 2. Recursive case - calls itself with a simpler input
    public static void countDown(int n) {
        if (n <= 0) {                // Base case
            System.out.println("Go!");
            return;
        }
        System.out.println(n);      // Work done at this level
        countDown(n - 1);          // Recursive call
    }

    public static void main(String[] args) {
        countDown(5);
        // Output: 5, 4, 3, 2, 1, Go!
    }
}
```

2 How Recursion Works – The Call Stack

Every time a method is invoked in Java, the JVM pushes a new **stack frame** onto the call stack. This frame contains the method's local variables, parameters, and return address. Recursive calls keep stacking frames until the base case is reached, after which the frames are popped (unwound) in reverse order, passing return values back up the chain.

Stack Frame	Content
factorial(1)	n=1 → returns 1 ← TOP (last called)
factorial(2)	n=2 → waits for factorial(1)
factorial(3)	n=3 → waits for factorial(2)
factorial(4)	n=4 → waits for factorial(3)
factorial(5)	n=5 → waits for factorial(4) ← BOTTOM (first called)
main()	Entry point ← STACK BASE

Warning: Each stack frame consumes memory. Very deep recursion (e.g., 10,000+ levels) causes a `java.lang.StackOverflowError`. Always ensure the base case is reachable.

3 Base Case and Recursive Case

Every correct recursive function must contain **two essential parts**:

- **Base Case** — The condition under which the function stops calling itself and returns a direct result. Without it, recursion would continue infinitely.
- **Recursive Case** — The part where the function calls itself with a simpler/smaller version of the original problem, progressing towards the base case.

[illegible]

4 Classic Recursion Examples

4.1 Factorial

The factorial of a non-negative integer n (written $n!$) is the product of all positive integers up to n . By definition, $0! = 1$. Factorial is the canonical introductory recursion example: $\text{factorial}(n) = n * \text{factorial}(n-1)$.

```
public class Factorial {
    public static long factorial(int n) {
        if (n < 0) throw new IllegalArgumentException("n must be >= 0");
        if (n == 0 || n == 1) return 1;           // Base case
        return (long) n * factorial(n - 1);       // Recursive case
    }

    public static void main(String[] args) {
        for (int i = 0; i <= 10; i++) {
            System.out.printf("%2d! = %d%n", i, factorial(i));
        }
    }
}

/* Output:
  0! = 1
  1! = 1
  2! = 2
  5! = 120
 10! = 3628800 */
```

4.2 Fibonacci Sequence

The Fibonacci sequence is defined as: $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$. It is a natural fit for recursion but also illustrates its inefficiency — naive recursion recomputes the same values many times (exponential time complexity).

```

public class Fibonacci {

    // Naive recursive -  $O(2^n)$  time
    public static long fibNaive(int n) {
        if (n <= 1) return n; // Base cases: F(0)=0, F(1)=1
        return fibNaive(n - 1) + fibNaive(n - 2); // Recursive case
    }

    // Memoized recursive -  $O(n)$  time
    private static long[] memo = new long[100];
    public static long fibMemo(int n) {
        if (n <= 1) return n;
        if (memo[n] != 0) return memo[n]; // Return cached value
        memo[n] = fibMemo(n - 1) + fibMemo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        System.out.println("F(10) = " + fibNaive(10)); // 55
        System.out.println("F(50) = " + fibMemo(50)); // 12586269025
    }
}

```

Tip: Always prefer memoization or iterative Fibonacci for large n . The naive version has $O(2^n)$ time complexity — fibNaive(50) makes ~2 trillion calls!

4.3 Sum of Natural Numbers

```

public static int sum(int n) {
    if (n == 0) return 0; // Base case
    return n + sum(n - 1); // Recursive case
}
// sum(5) = 5 + 4 + 3 + 2 + 1 + 0 = 15

```

4.4 Power Function

Computing base^{exp} recursively. An optimised version uses the fact that $x^n = (x^{n/2})^2$ for even n , giving $O(\log n)$ time.


```
public class Power {  
    // Simple O(n) version  
    public static double power(double base, int exp) {  
        if (exp == 0) return 1;           //  $x^0 = 1$   
        if (exp < 0) return 1.0 / power(base, -exp);  
        return base * power(base, exp - 1);  
    }  
  
    // Fast exponentiation - O(log n)  
    public static double fastPower(double base, int exp) {  
        if (exp == 0) return 1;  
        if (exp % 2 == 0) {  
            double half = fastPower(base, exp / 2);  
            return half * half;           // Avoid recomputing  
        }  
        return base * fastPower(base, exp - 1);  
    }  
}
```

5 Recursion with Arrays & Strings

5.1 Reverse a String

```
public static String reverse(String s) {
    if (s.isEmpty() || s.length() == 1) return s; // Base case
    // Take last char + reverse of remaining string
    return s.charAt(s.length() - 1) + reverse(s.substring(0, s.length() - 1));
}
// reverse("hello") -> "olleh"
```

5.2 Binary Search (Recursive)

Binary search on a sorted array repeatedly halves the search space — a natural divide-and-conquer recursive pattern with $O(\log n)$ time complexity.

```
public static int binarySearch(int[] arr, int target, int low, int high) {
    if (low > high) return -1; // Base case: not found

    int mid = low + (high - low) / 2; // Prevent integer overflow

    if (arr[mid] == target) return mid; // Base case: found
    if (arr[mid] < target)
        return binarySearch(arr, target, mid + 1, high); // Search right half
    else
        return binarySearch(arr, target, low, mid - 1); // Search left half
}

// Usage:
// int[] sorted = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
// int idx = binarySearch(sorted, 23, 0, sorted.length - 1); // returns 5
```

5.3 Array Sum

```
public static int arraySum(int[] arr, int index) {
    if (index == arr.length) return 0; // Base case: past the end
    return arr[index] + arraySum(arr, index + 1); // Add current + rest
}
// int[] nums = {1, 2, 3, 4, 5};
// arraySum(nums, 0) -> 15
```

6 Advanced Recursion Patterns

6.1 Tower of Hanoi

A classic puzzle with 3 pegs. Move n discs from source to destination using an auxiliary peg, never placing a larger disc on a smaller one. Solving it requires exactly $2^n - 1$ moves.

```
public class TowerOfHanoi {
    public static void hanoi(int n, char from, char to, char aux) {
        if (n == 1) {
            System.out.println("Move disc 1 from " + from + " to " + to);
            return;
        }
        hanoi(n - 1, from, aux, to); // Move n-1 discs to aux
        System.out.println("Move disc " + n + " from " + from + " to " + to);
        hanoi(n - 1, aux, to, from); // Move n-1 discs from aux to dest
    }

    public static void main(String[] args) {
        hanoi(3, 'A', 'C', 'B');
        // 7 moves total for n=3
    }
}
```

6.2 Merge Sort

Merge Sort is a classic divide-and-conquer algorithm. It recursively splits an array in half, sorts each half, then merges them. Time complexity: $O(n \log n)$.

```

public class MergeSort {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left >= right) return;           // Base case: 1 element

        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);           // Sort left half
        mergeSort(arr, mid + 1, right);      // Sort right half
        merge(arr, left, mid, right);        // Merge sorted halves
    }

    private static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1, n2 = right - mid;
        int[] L = new int[n1], R = new int[n2];
        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);
        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2)
            arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }
}

```

6.3 Binary Tree Traversal

Tree structures are inherently recursive — each node contains sub-trees. All three classic traversals (inorder, preorder, postorder) use recursion naturally.

```

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class TreeTraversal {
    // InOrder: Left -> Root -> Right (gives sorted output for BST)
    public static void inOrder(TreeNode node) {
        if (node == null) return;          // Base case
        inOrder(node.left);
        System.out.print(node.val + " ");
        inOrder(node.right);
    }

    // PreOrder: Root -> Left -> Right
    public static void preOrder(TreeNode node) {
        if (node == null) return;
        System.out.print(node.val + " ");
        preOrder(node.left);
        preOrder(node.right);
    }

    // PostOrder: Left -> Right -> Root
    public static void postOrder(TreeNode node) {
        if (node == null) return;
        postOrder(node.left);
        postOrder(node.right);
        System.out.print(node.val + " ");
    }
}

```

6.4 Backtracking – N-Queens Problem

Backtracking is a recursive algorithmic technique that builds solutions incrementally and abandons ('backtracks') partial solutions that cannot lead to a valid answer. The N-Queens problem places N queens on an N×N board so no two queens attack each other.

```

public class NQueens {
    public static void solve(int[][] board, int row, int n) {
        if (row == n) { // Base case: all queens placed
            printBoard(board, n);
            return;
        }
        for (int col = 0; col < n; col++) {
            if (isSafe(board, row, col, n)) {
                board[row][col] = 1; // Place queen
                solve(board, row + 1, n); // Recurse to next row
                board[row][col] = 0; // Backtrack (undo placement)
            }
        }
    }

    private static boolean isSafe(int[][] b, int r, int c, int n) {
        for (int i = 0; i < r; i++) if (b[i][c] == 1) return false;
        for (int i=r-1, j=c-1; i>=0 && j>=0; i--,j--) if (b[i][j]==1) return false;
        for (int i=r-1, j=c+1; i>=0 && j<n; i--,j++) if (b[i][j]==1) return false;
        return true;
    }

    private static void printBoard(int[][] b, int n) {
        for (int[] row : b) {
            for (int cell : row) System.out.print((cell == 1 ? "Q" : ".") + " ");
            System.out.println();
        }
        System.out.println();
    }
}

```

7 Recursion vs Iteration

Any recursive algorithm can be implemented iteratively and vice versa. Choosing between them involves trade-offs:

Aspect	Recursion	Iteration
Readability	Often cleaner for tree/graph problems	Cleaner for simple loops
Performance	Extra stack overhead per call	Generally faster, no overhead
Memory	$O(n)$ stack frames	$O(1)$ extra space (usually)
Debugging	Harder to trace deep calls	Easier to step through
Stack Overflow	Risk with deep recursion	No risk
Code length	Usually shorter	Can be verbose
Best for	Trees, graphs, D&C, backtracking	Simple loops, performance-critical

Iterative vs Recursive Factorial — Side by Side

```
// Iterative                                // Recursive
long factIter(int n) {                      long factRec(int n) {
    long result = 1;                        if (n <= 1) return 1;
    for (int i = 2; i <= n; i++)             return n * factRec(n - 1);
        result *= i;                        }
    return result;
}
//  $O(n)$  time,  $O(1)$  space                //  $O(n)$  time,  $O(n)$  stack space
```

8 Common Mistakes & Best Practices

Missing Base Case

The most common error. Without a proper base case, recursion never terminates.

```
// WRONG - infinite recursion!
public static int countDown(int n) {
    System.out.println(n);
    return countDown(n - 1); // No base case!
}

// CORRECT
public static int countDown(int n) {
    if (n <= 0) return 0; // Base case
    System.out.println(n);
    return countDown(n - 1);
}
```

Not Moving Towards Base Case

The recursive call must always reduce the problem. If *n* stays the same or grows, you have infinite recursion.

```
// WRONG - n never decreases
public static int bad(int n) {
    if (n == 0) return 0;
    return bad(n); // Same n! Infinite loop
}
```

Stack Overflow for Large Inputs

For very large inputs, use tail-call optimization (not natively supported in Java), memoization, or convert to iteration.

```
// For large n, use iteration or memoization instead
// Java does NOT optimise tail calls automatically
public static long safeFactorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) result *= i; // Iterative - safe
    return result;
}
```

Best Practices Summary

- ✓ Always define a clear, reachable **base case** before writing the recursive call.

- ✓ Ensure each recursive call brings the input **closer to the base case**.
- ✓ For repeated sub-problems, use **memoization** (top-down DP) to cache results.
- ✓ Test with the smallest valid inputs ($n=0$, $n=1$) first.
- ✓ Prefer iteration for simple linear problems; use recursion for tree/graph/D&C; algorithms.
- ✓ Be mindful of **stack depth limits** — default Java stack is ~500–1000 frames deep.
- ✓ Use helper methods with extra parameters (e.g., index, accumulator) to simplify signatures.
- ✓ Add **input validation** (IllegalArgumentException) at the entry point, not inside recursion.

Recursion is a fundamental tool in every Java developer's toolkit. Master the base-case discipline and the divide-and-conquer mindset, and you will find elegant, readable solutions to problems that would otherwise require complex iterative logic.